D20A-52 **Experiment No: 2**

AIM: Create a Blockchain using Python

Lab Objectives: To implement public and private Blockchain.

***Lab Outcomes (LO)*:** Demonstrate the concept of Blockchain in real-world Applications (LO4)

Task to be performed :

1. Make a copy of this **Google Colab Notebook**
2. Try to solve the errors in given Program
3. After successful execution of the Program in Colab Notebook.
    - **Add a method, create_Transactions**
    - Mine the block only when the transaction list is not null.
    - Remove the transactions from the list of transactions before mining.
    - **Modify the method, proof_of_work() to search for the golden nonce**
    - Cryptographic Puzzle is to have **"000" leading zeros** in the Block Hash
4. Download the code - **blockchain.py**
5. **Update the code to incorporate the changes in step 3 to the code in step 4.**
6. Follow the steps in **Manual** to demonstrate the working of Blockchain using Flask and Postman.

Tools & Libraries used :

- ***Flask*** : pip install Flask
- Download ***Postman*** from https://www.postman.com/
- Python Libraries : **datetime, jsonify, hashlib**

Instructions : (Prepare for viva for the following topics)

1. What is a Blockchain?
2. Process of Mining
3. How to check the validity of blocks in a Blockchain

Outcome :

1. Understood the concept of mining in Blockchain.
2. Implemented programs
    - Mine a Block
    - Display the chain in the Blockchain
    - Check whether the chain is valid
3. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.
4. Submit the hardcopy **by the 1st week of Sem**

   (As per the instructions, submit a hard copy of the same).

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.

2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).

3. **Hashing** – SHA-256 algorithm ensures data immutability.

4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.

5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

1. Importing Required Libraries

- **datetime** → to timestamp each block

- **hashlib** → to generate SHA-256 hashes

- **json** → to encode blocks as JSON for hashing

- **Flask** → to run the blockchain API server

2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
- store the chain
- implement Proof of Work
- validate chain integrit

3. Flask Web API

Provides endpoints:

- `/mine_block` → mine a new block
- `/get_chain` → fetch full blockchain
- `/is_valid` → verify blockchain

COMPLETE CODE WITH COMMENTS (COPY-PASTE READY)

VS CODE Blockchain.py:

```
#
==========================================================================================
==============
# BLOCKCHAIN LAB - EXPERIMENT 2
# Create a Blockchain using Python
#
==========================================================

# Install Flask before running:
# pip install flask

# Importing required libraries
import datetime
import hashlib
import json

from flask import Flask, jsonify, request
```

```
============
'ART 1 - BUILDING A BLOCKCHAIN

==========================================
==============

ss Blockchain:

♯ ----------------------------------------------------
♯ Initialize Blockchain
♯ ----------------------------------------------------
lef __init__(self):

  # Store all blocks
  self.chain = []

  # Store pending transactions
  self.transactions = []
```

```
    # Create Genesis Block
    self.create_block(
        proof=1,
        previous_hash='0',
        transactions=[]
    )

# ------------------------------------------------------
# Create a Block
# ------------------------------------------------------
def create_block(self, proof, previous_hash, transactions):

    block = {
        'index': len(self.chain) + 1,
        'timestamp': str(datetime.datetime.now()),
        'proof': proof,
        'previous_hash': previous_hash,
        'transactions': transactions
    }

    # Add block to blockchain
    self.chain.append(block)

    return block

# ------------------------------------------------------
# Create Transactions
# ------------------------------------------------------
def create_transactions(self, sender, receiver, amount):

    transaction = {
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    }

    # Add transaction to pending transaction list
    self.transactions.append(transaction)

    # Return the block number in which
    # this transaction will be stored
    return len(self.chain) + 1

# ------------------------------------------------------
# Get Previous Block
# ------------------------------------------------------
def get_previous_block(self):

    return self.chain[-1]

# ------------------------------------------------------
# Proof of Work
# ------------------------------------------------------
def proof_of_work(self, previous_proof):

    new_proof = 1
    check_proof = False

    while check_proof is False:

        # Calculate hash for the current proof
        hash_operation = hashlib.sha256(
            str(
                new_proof ** 2 - previous_proof ** 2
            ).encode()
        ).hexdigest()

        # Cryptographic puzzle:
        # Hash must have 3 leading zeros
        if hash_operation[:3] == '000':

            check_proof = True

        else:

            new_proof += 1
```

```
    # Golden nonce
    return new_proof

# ---------------------------------------------------------
# Hash a Block
# ---------------------------------------------------------
def hash(self, block):

    # Convert block into JSON
    encoded_block = json.dumps(
        block,
        sort_keys=True
    ).encode()

    # Generate SHA-256 hash
    return hashlib.sha256(encoded_block).hexdigest()

# ---------------------------------------------------------
# Check Whether Blockchain is Valid
# ---------------------------------------------------------
def is_chain_valid(self, chain):

    # Start with Genesis Block
    previous_block = chain[0]

    block_index = 1

    while block_index < len(chain):

        block = chain[block_index]

        # Check whether previous hash is correct
        if block['previous_hash'] != self.hash(previous_block

            return False

        # Get proofs
        previous_proof = previous_block['proof']
        proof = block['proof']

        # Calculate Proof-of-Work hash
        hash_operation = hashlib.sha256(
            str(
                proof ** 2 - previous_proof ** 2
            ).encode()
        ).hexdigest()

        # Hash must have 3 leading zeros
        if hash_operation[:3] != '000':

            return False

        previous_block = block

        block_index += 1

    return True

# ============================================================
# PART 2 - FLASK WEB APPLICATION
# ============================================================

# Create Flask application
app = Flask(__name__)

# Create Blockchain object
blockchain = Blockchain()

# ============================================================
# ADD TRANSACTION
```

```
#
==========

@app.route('/add_transaction', methods=['POST'])
def add_transaction():

    # Get JSON data from Postman
    json_data = request.get_json()

    # Get transaction details
    sender = json_data['sender']
    receiver = json_data['receiver']
    amount = json_data['amount']

    # Create transaction
    block_index = blockchain.create_transactions(
        sender,
        receiver,
        amount
    )

    # Prepare response
    response = {
        'message': 'Transaction added successfully.',
        'sender': sender,
        'receiver': receiver,
        'amount': amount,
        'block_index': block_index
    }

    return jsonify(response), 201


#
==========
# MINE A NEW BLOCK
#
==========

app.route('/mine_block', methods=['GET'])
mine_block():

# ------------------------------------------------------
# Requirement:
# Mine only when transaction list is not empty
# ------------------------------------------------------

f len(blockchain.transactions) == 0:

    return jsonify({
        'message': 'No transactions available. Add a transaction
ore mining.'
    }), 400

# ------------------------------------------------------
# Copy the pending transactions
# ------------------------------------------------------

ransactions_to_mine = blockchain.transactions.copy()

# ------------------------------------------------------
# Remove transactions from pending list BEFORE mining
# ------------------------------------------------------

lockchain.transactions = []

# ------------------------------------------------------
# Get previous block
# ------------------------------------------------------

revious_block = blockchain.get_previous_block()

revious_proof = previous_block['proof']

# ------------------------------------------------------
```

```
    # Find Golden Nonce using Proof of Work
    # ------------------------------------------------------

    proof = blockchain.proof_of_work(previous_proof)

    # ------------------------------------------------------
    # Get hash of previous block
    # ------------------------------------------------------

    previous_hash = blockchain.hash(previous_block)

    # ------------------------------------------------------
    # Create new block
    # ------------------------------------------------------

    block = blockchain.create_block(
        proof,
        previous_hash,
        transactions_to_mine
    )

    # ------------------------------------------------------
    # Prepare response
    # ------------------------------------------------------

    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }

    return jsonify(response), 200

#
==========
GET THE FULL BLOCKCHAIN
==========
pp.route('/get_chain', methods=['GET'])
get_chain():

esponse = {
    'chain': blockchain.chain,
    'length': len(blockchain.chain)
}

eturn jsonify(response), 200

==========
CHECK WHETHER BLOCKCHAIN IS VALID
==========
pp.route('/is_valid', methods=['GET'])
is_valid():

alid = blockchain.is_chain_valid(
    blockchain.chain
)

f valid:

    response = {
        'message': 'All good. The Blockchain is valid.'
    }
==========lse:
```

```
        response = {
            'message': 'Houston, we have a problem. The Block
is not valid.'
        }

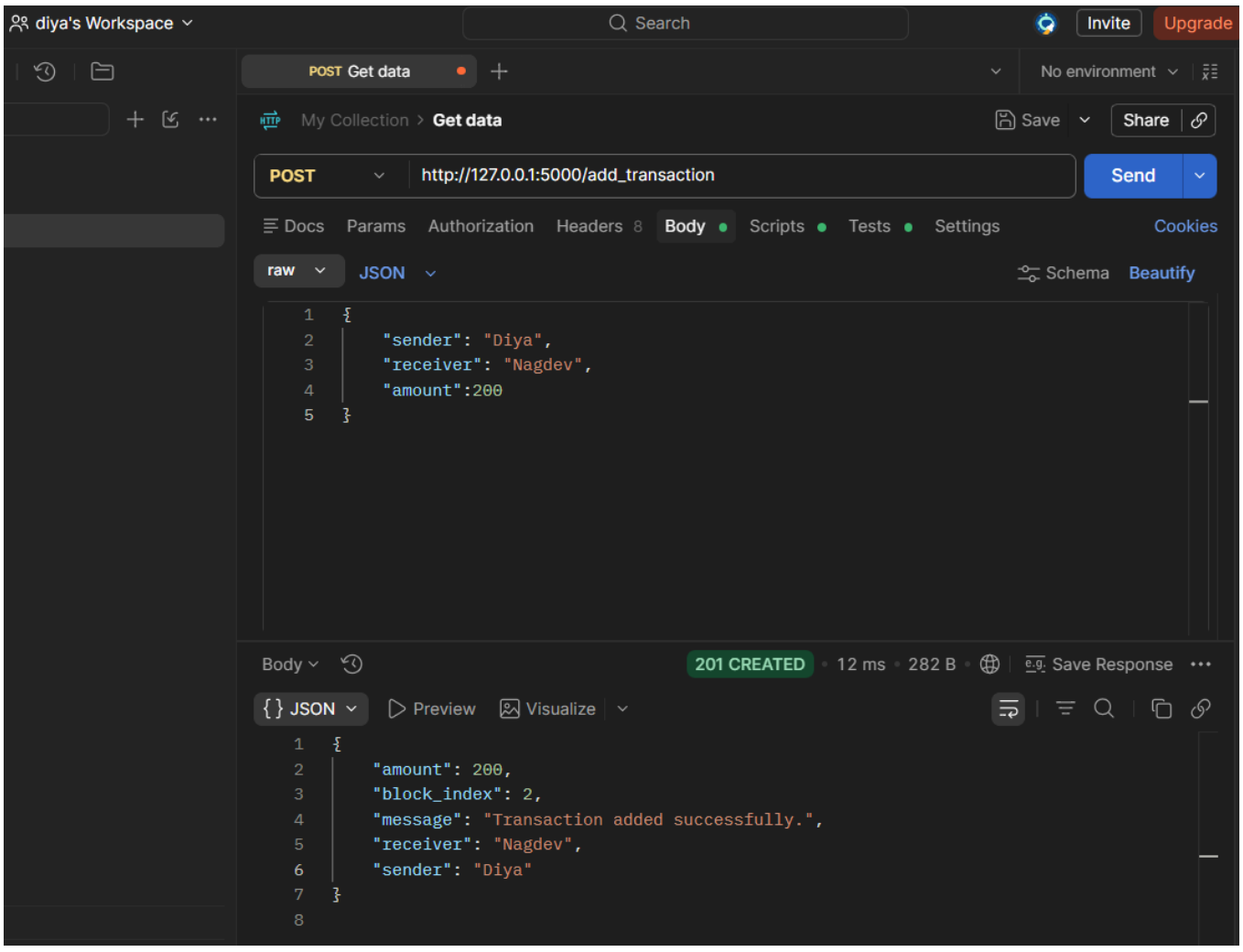
    return jsonify(response), 200
```

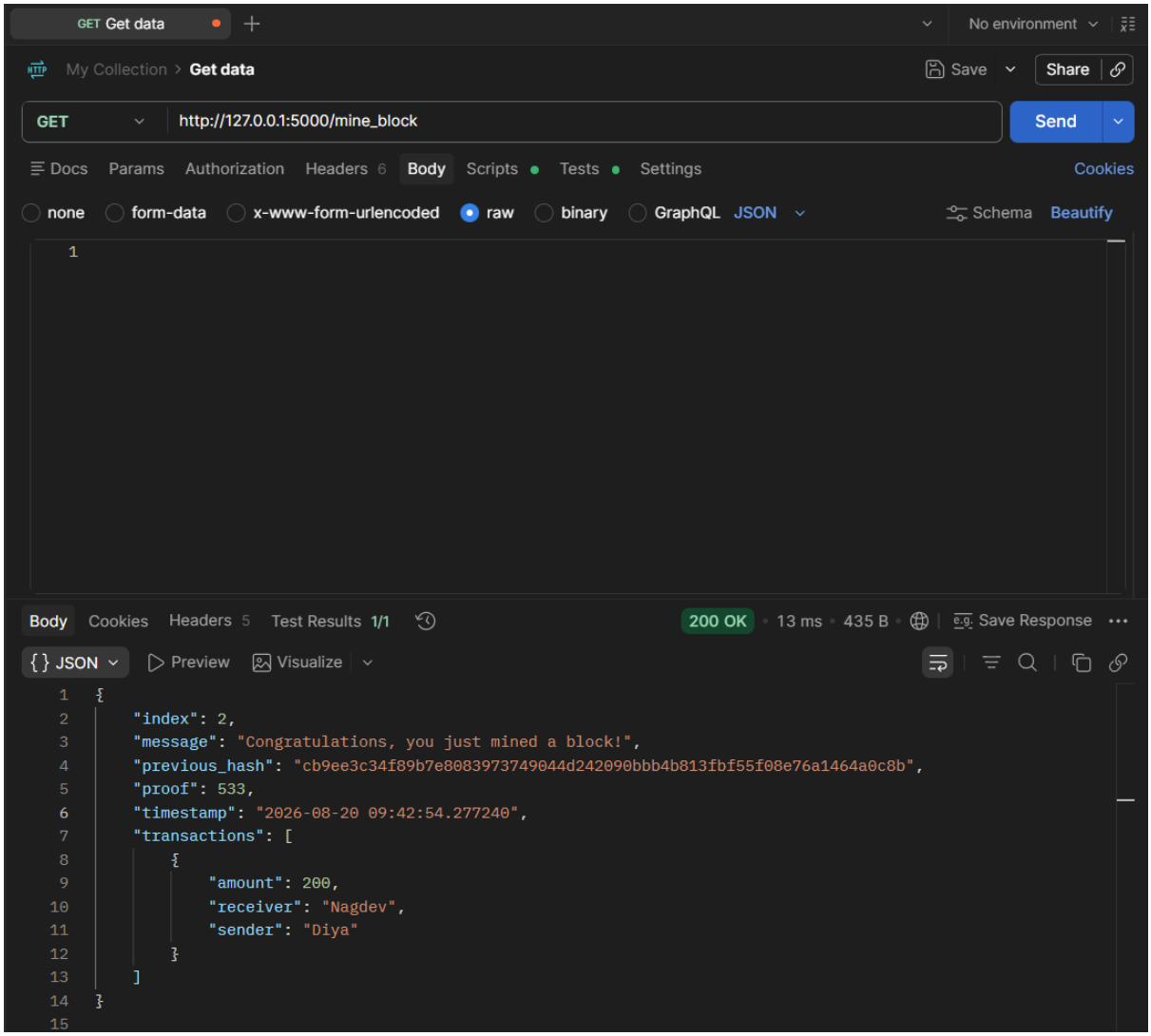
```
============
:UN FLASK APPLICATION
==============================================
============

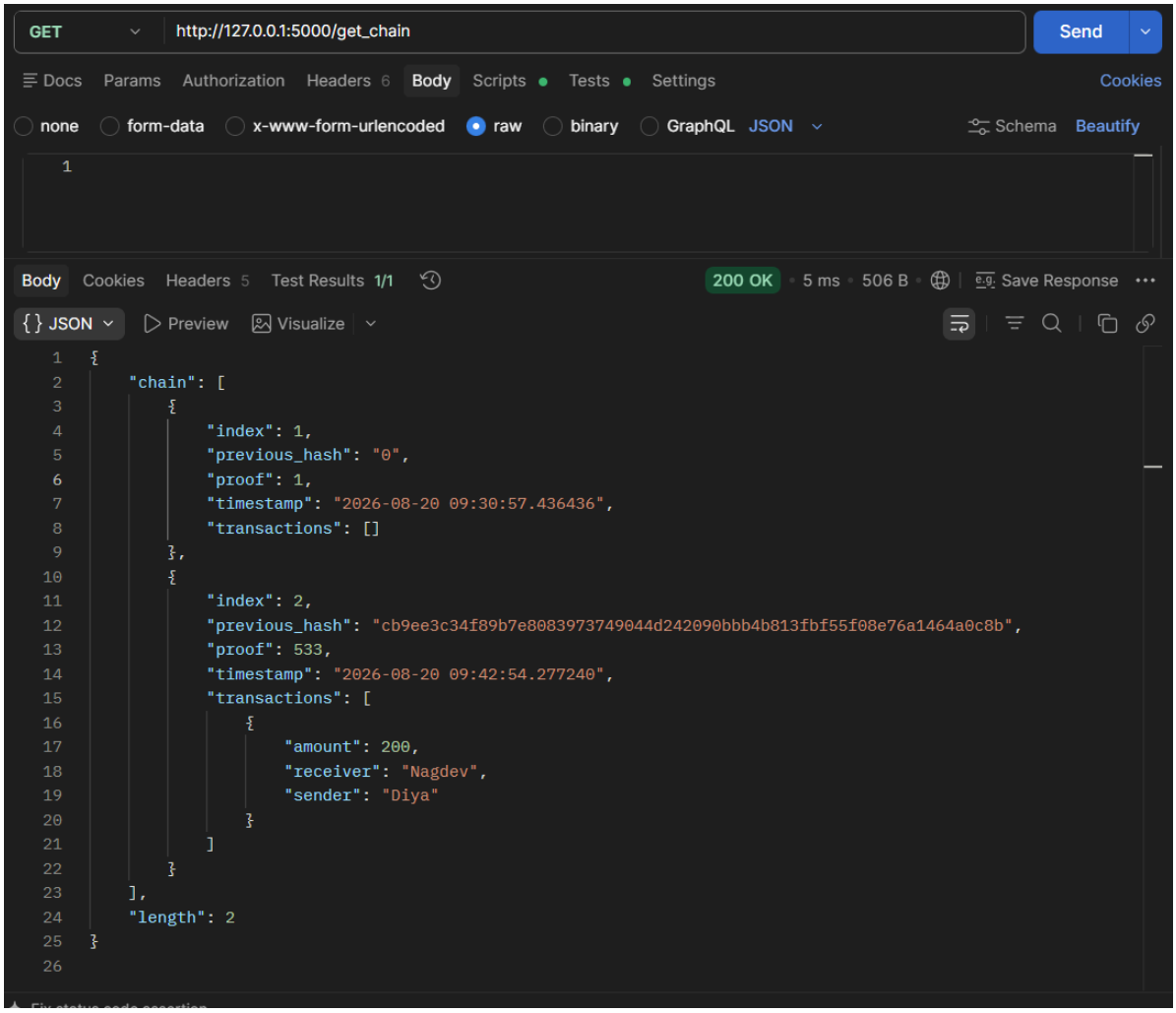
).run(
host='0.0.0.0',
port=5000
```

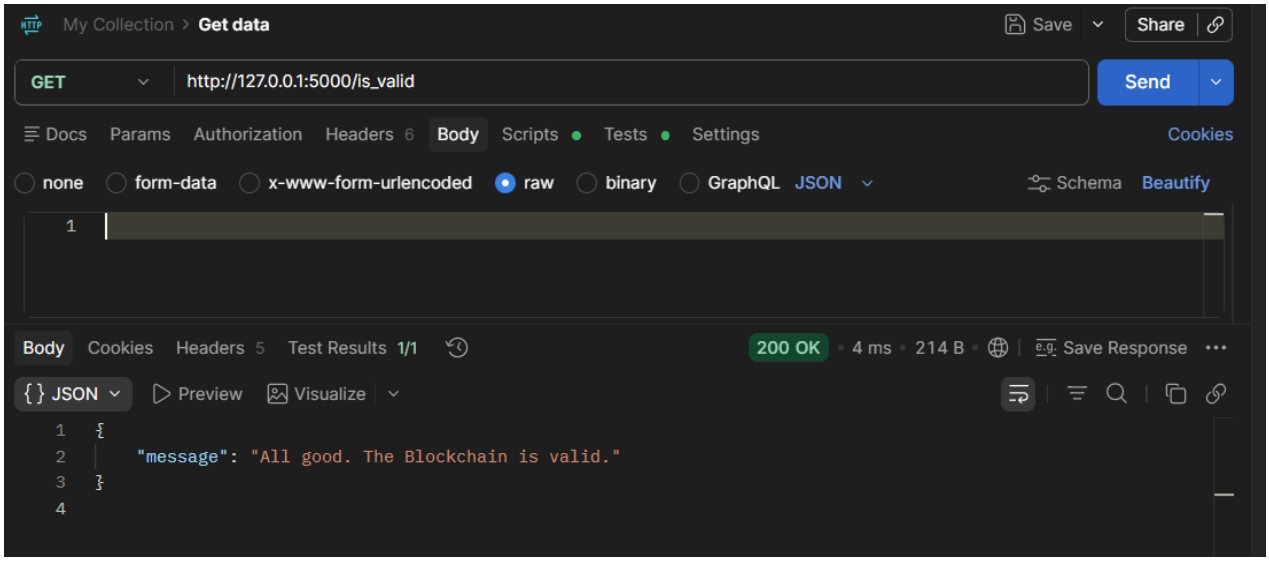
```
#
==============================================
```

Transaction - 1

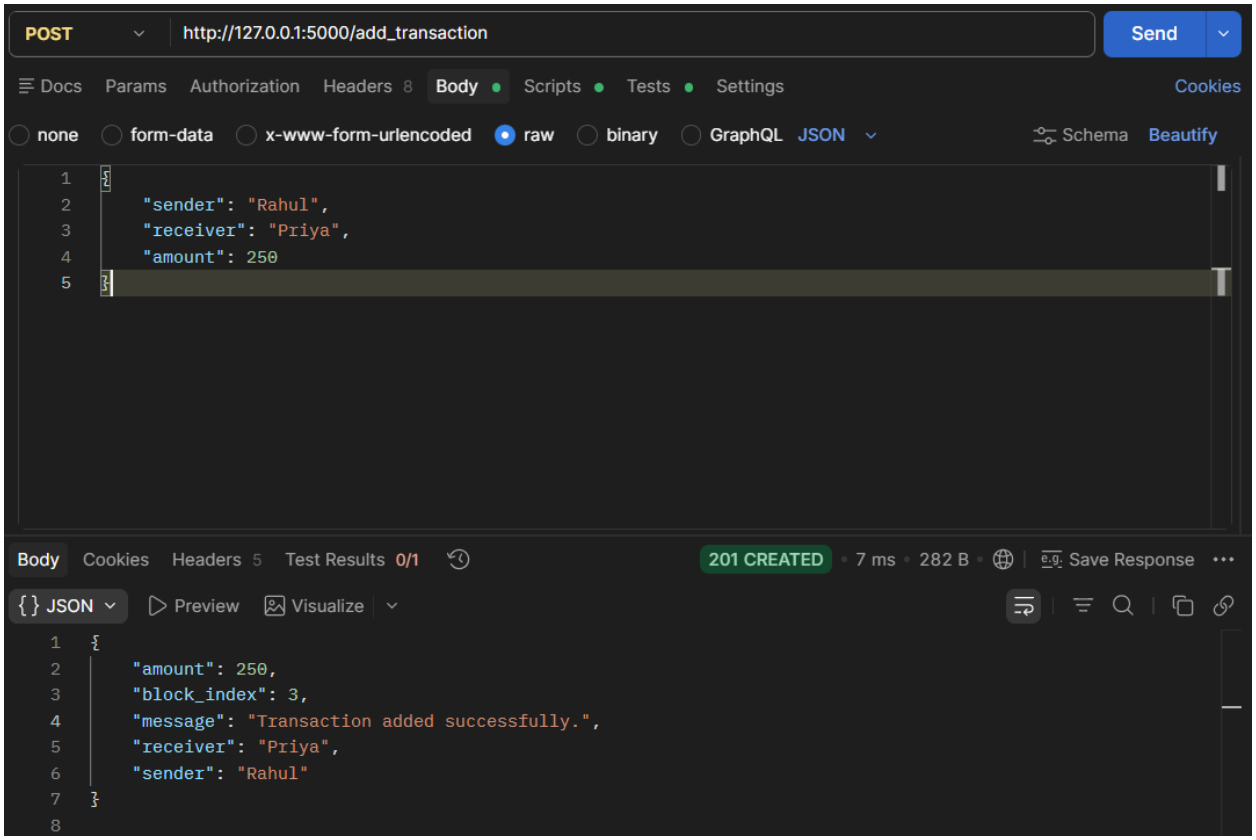


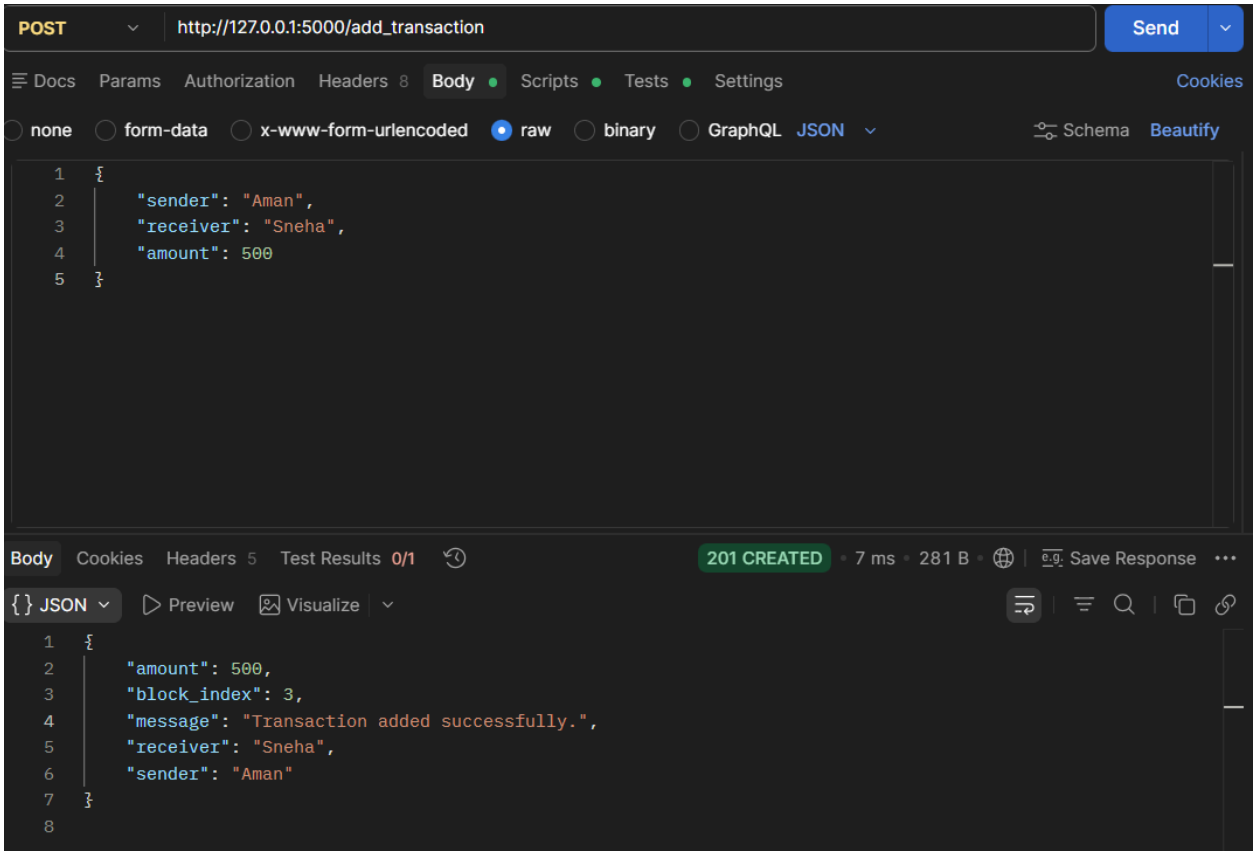

GET Get data
No environment
My Collection > Get data
Save
Share
GET
http://127.0.0.1:5000/mine_block
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
1
Body
Cookies
Headers 5
Test Results 1/1
200 OK
13 ms
435 B
Save Response
{} JSON
Preview
Visualize
{
"index": 2,
"message": "Congratulations, you just mined a block!",
"previous_hash": "cb9ee3c34f89b7e8083973749044d242090bbb4b813fbf55f08e76a1464a0c8b",
"proof": 533,
"timestamp": "2026-08-20 09:42:54.277240",
"transactions": [
{
"amount": 200,
"receiver": "Nagdev",
"sender": "Diya"
}
]
}





[Transaction - 2]


POST
http://127.0.0.1:5000/add_transaction
Send
Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"sender": "Rahul",
"receiver": "Priya",
"amount": 250
}
Body Cookies Headers 5 Test Results 0/1
201 CREATED 7 ms 282 B Save Response
{ } JSON Preview Visualize
{
"amount": 250,
"block_index": 3,
"message": "Transaction added successfully.",
"receiver": "Priya",
"sender": "Rahul"
}


POST
http://127.0.0.1:5000/add_transaction
Send
Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"sender": "Aman",
"receiver": "Sneha",
"amount": 500
}
Body Cookies Headers 5 Test Results 0/1
201 CREATED 7 ms 281 B Save Response
{ } JSON Preview Visualize
{
"amount": 500,
"block_index": 3,
"message": "Transaction added successfully.",
"receiver": "Sneha",
"sender": "Aman"
}

GET http://127.0.0.1:5000/mine_block Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cook

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1
```

Body Cookies Headers 5 Test Results 1/1 200 OK 42 ms 487 B Save Response

{ } JSON Preview Visualize


```
{
    "index": 3,
    "message": "Congratulations, you just mined a block!",
    "previous_hash": "3dee07b3cedea9a4baacf527009ac3ae80e89a0282f7ac23a42ceadebd05f0e1",
    "proof": 11028,
    "timestamp": "2026-08-20 09:52:59.052321",
    "transactions": [
        {
            "amount": 250,
            "receiver": "Priya",
            "sender": "Rahul"
        },
        {
            "amount": 500,
            "receiver": "Sneha",
            "sender": "Aman"
        }
    ]
}
```


GET http://127.0.0.1:5000/get_chain Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

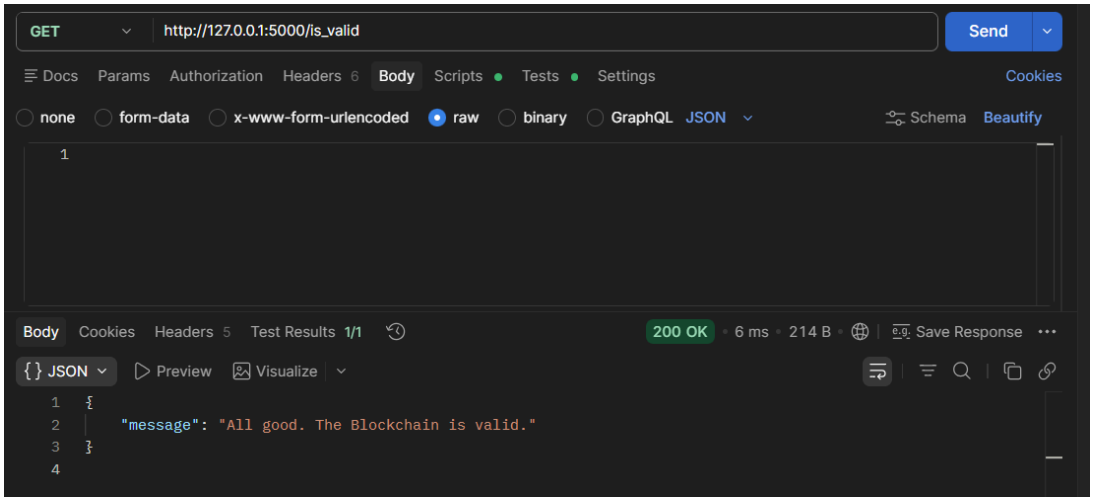
```
1
```

Body Cookies Headers 5 Test Results 1/1 200 OK 8 ms 1.01 KB Save Response

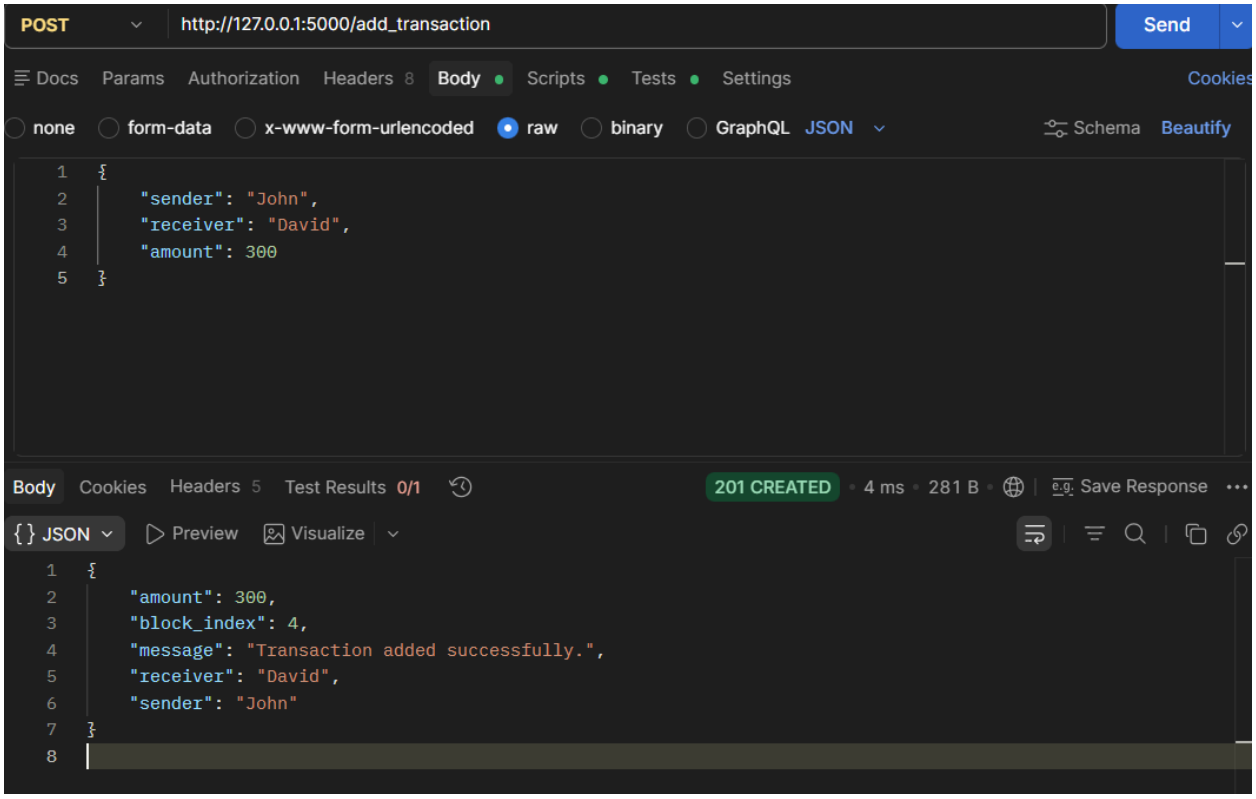
{ } JSON Preview Visualize

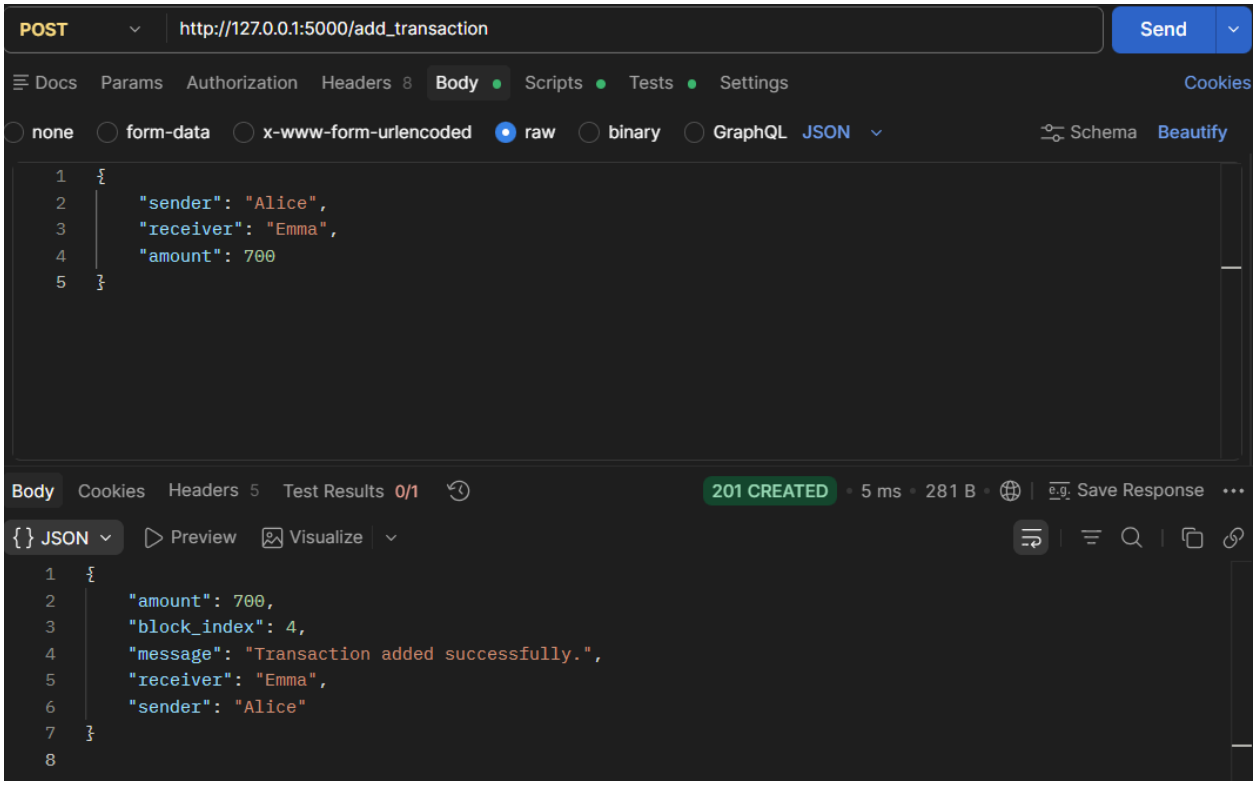

```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "0",
            "proof": 1,
            "timestamp": "2026-08-20 09:30:57.436436",
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "cb9ee3c34f89b7e8083973749044d242090bbb4b813fbf55f08e76a1464a0c8b",
            "proof": 533,
            "timestamp": "2026-08-20 09:42:54.277240",
            "transactions": [
                {
                    "amount": 200,
                    "receiver": "Nagdev",
                    "sender": "Diya"
                }
            ]
        },
        {
            "index": 3,
            "previous_hash": "3dee07b3cedea9a4baacf527009ac3ae80e89a0282f7ac23a42ceadebd05f0e1",
            "proof": 11028,
            "timestamp": "2026-08-20 09:52:59.052321",
            "transactions": [
                {
```

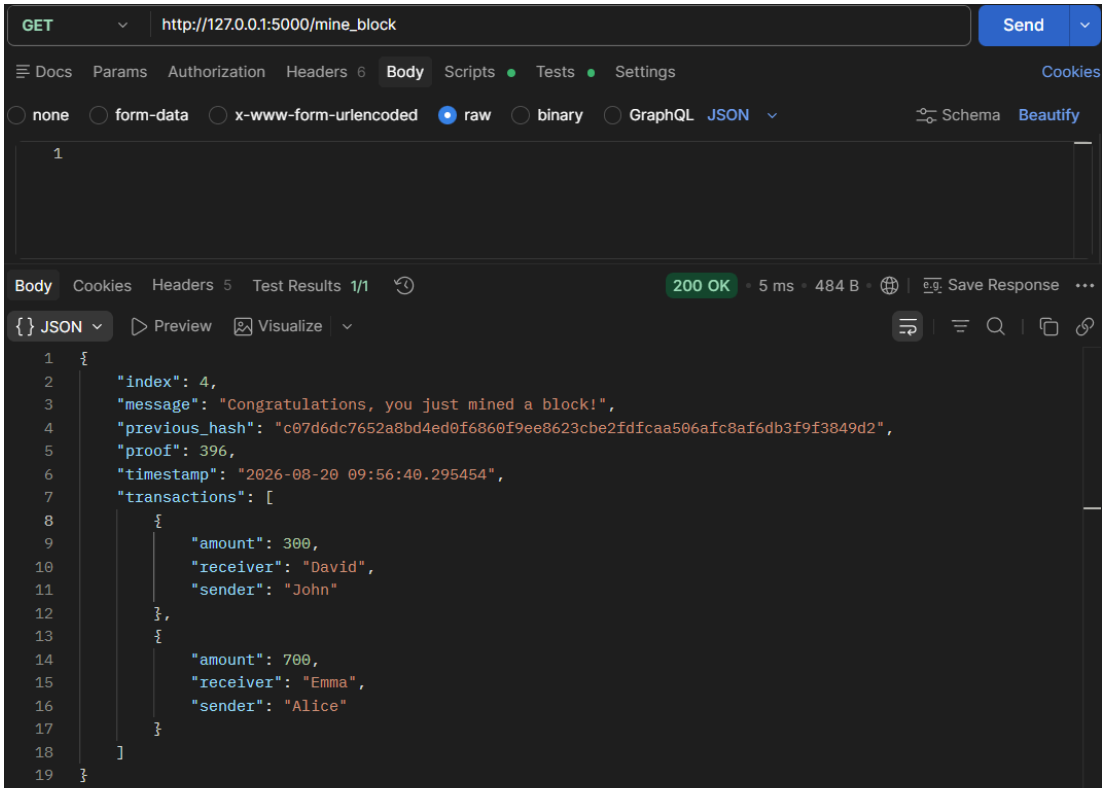
```
                {
                    "amount": 500,
                    "receiver": "Sneha",
                    "sender": "Aman"
                }
            ]
        },
        {
            "index": 4,
            "previous_hash": "c07d6dc7652a8bd4ed0f6860f9ee8623cbe2fdfcaa506afc8af6db3f9f3849d2",
            "proof": 396,
            "timestamp": "2026-08-20 09:56:40.295454",
            "transactions": [
                {
                    "amount": 300,
                    "receiver": "David",
                    "sender": "John"
                },
                {
                    "amount": 700,
                    "receiver": "Emma",
                    "sender": "Alice"
                }
            ]
        }
    ],
    "length": 4
}
```

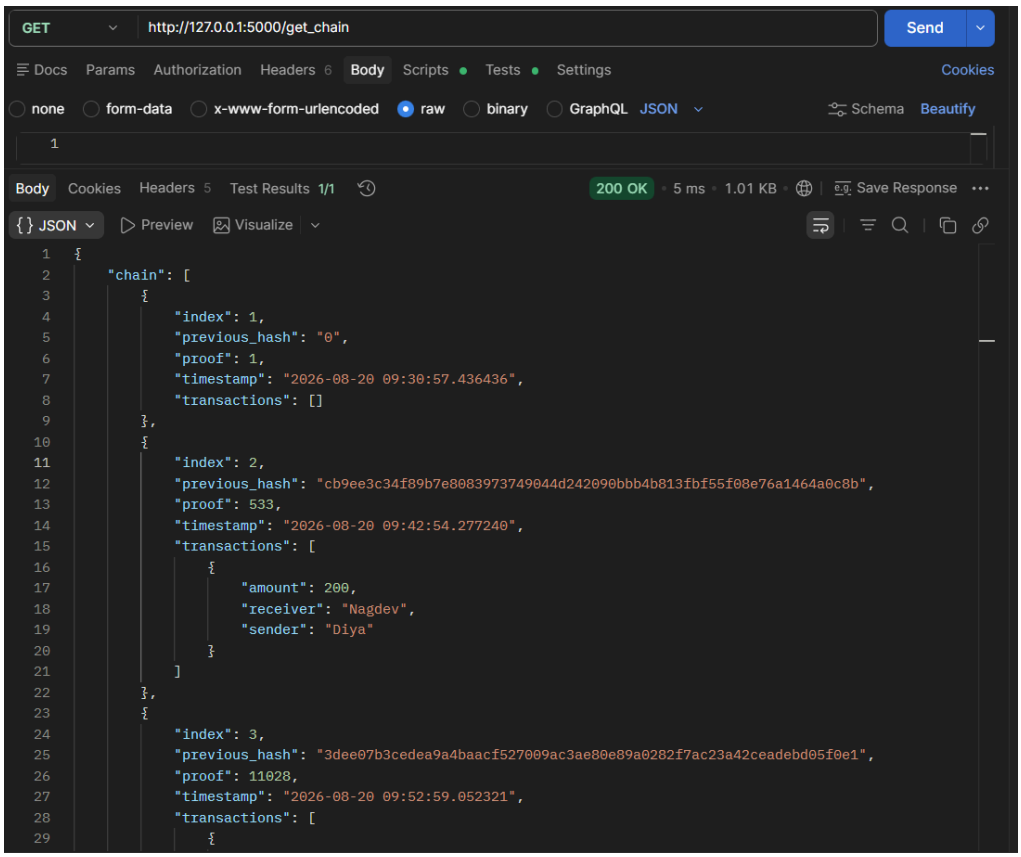



[Transaction - 3]

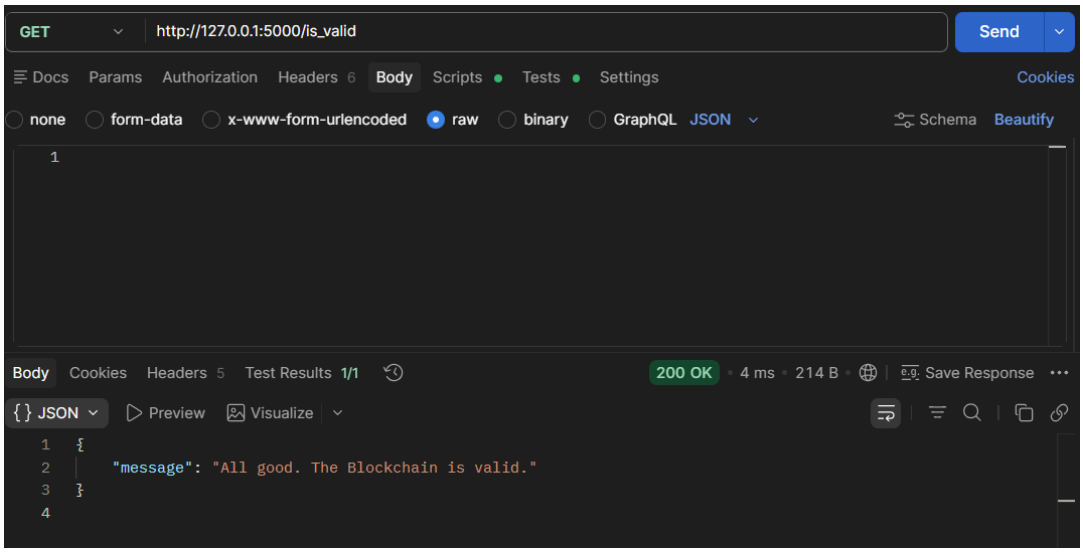

POST
http://127.0.0.1:5000/add_transaction
Send
Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"sender": "John",
"receiver": "David",
"amount": 300
}
Body Cookies Headers 5 Test Results 0/1
201 CREATED 4 ms 281 B Save Response
{ } JSON Preview Visualize
{
"amount": 300,
"block_index": 4,
"message": "Transaction added successfully.",
"receiver": "David",
"sender": "John"
}


POST
http://127.0.0.1:5000/add_transaction
Send
Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"sender": "Alice",
"receiver": "Emma",
"amount": 700
}
Body Cookies Headers 5 Test Results 0/1
201 CREATED 5 ms 281 B Save Response
{ } JSON Preview Visualize
{
"amount": 700,
"block_index": 4,
"message": "Transaction added successfully.",
"receiver": "Emma",
"sender": "Alice"
}


GET
http://127.0.0.1:5000/mine_block
Send
Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify
1
Body Cookies Headers 5 Test Results 1/1 200 OK 5 ms 484 B Save Response
{ } JSON Preview Visualize
{
"index": 4,
"message": "Congratulations, you just mined a block!",
"previous_hash": "c07d6dc7652a8bd4ed0f6860f9ee8623cbe2fdfcaa506afc8af6db3f9f3849d2",
"proof": 396,
"timestamp": "2026-08-20 09:56:40.295454",
"transactions": [
{
"amount": 300,
"receiver": "David",
"sender": "John"
},
{
"amount": 700,
"receiver": "Emma",
"sender": "Alice"
}
]
}


GET
http://127.0.0.1:5000/get_chain
Send
Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify
1
Body Cookies Headers 5 Test Results 1/1 200 OK 5 ms 1.01 KB Save Response
{ } JSON Preview Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-20 09:30:57.436436",
"transactions": []
},
{
"index": 2,
"previous_hash": "cb9ee3c34f89b7e8083973749044d242090bbb4b813fbf55f08e76a1464a0c8b",
"proof": 533,
"timestamp": "2026-08-20 09:42:54.277240",
"transactions": [
{
"amount": 200,
"receiver": "Nagdev",
"sender": "Diya"
}
]
},
{
"index": 3,
"previous_hash": "3dee07b3cedea9a4baacf527009ac3ae80e89a0282f7ac23a42ceadebd05f0e1",
"proof": 11028,
"timestamp": "2026-08-20 09:52:59.052321",
"transactions": [
{

```
                },
                {
                    "amount": 500,
                    "receiver": "Sneha",
                    "sender": "Aman"
                }
            ]
        },
        {
            "index": 4,
            "previous_hash": "c07d6dc7652a8bd4ed0f6860f9ee8623cbe2fdfcaa506afc8af6db3f9f3849d2",
            "proof": 396,
            "timestamp": "2026-08-20 09:56:40.295454",
            "transactions": [
                {
                    "amount": 300,
                    "receiver": "David",
                    "sender": "John"
                },
                {
                    "amount": 700,
                    "receiver": "Emma",
                    "sender": "Alice"
                }
            ]
        }
    ],
    "length": 4
}
```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.